

ООП 1

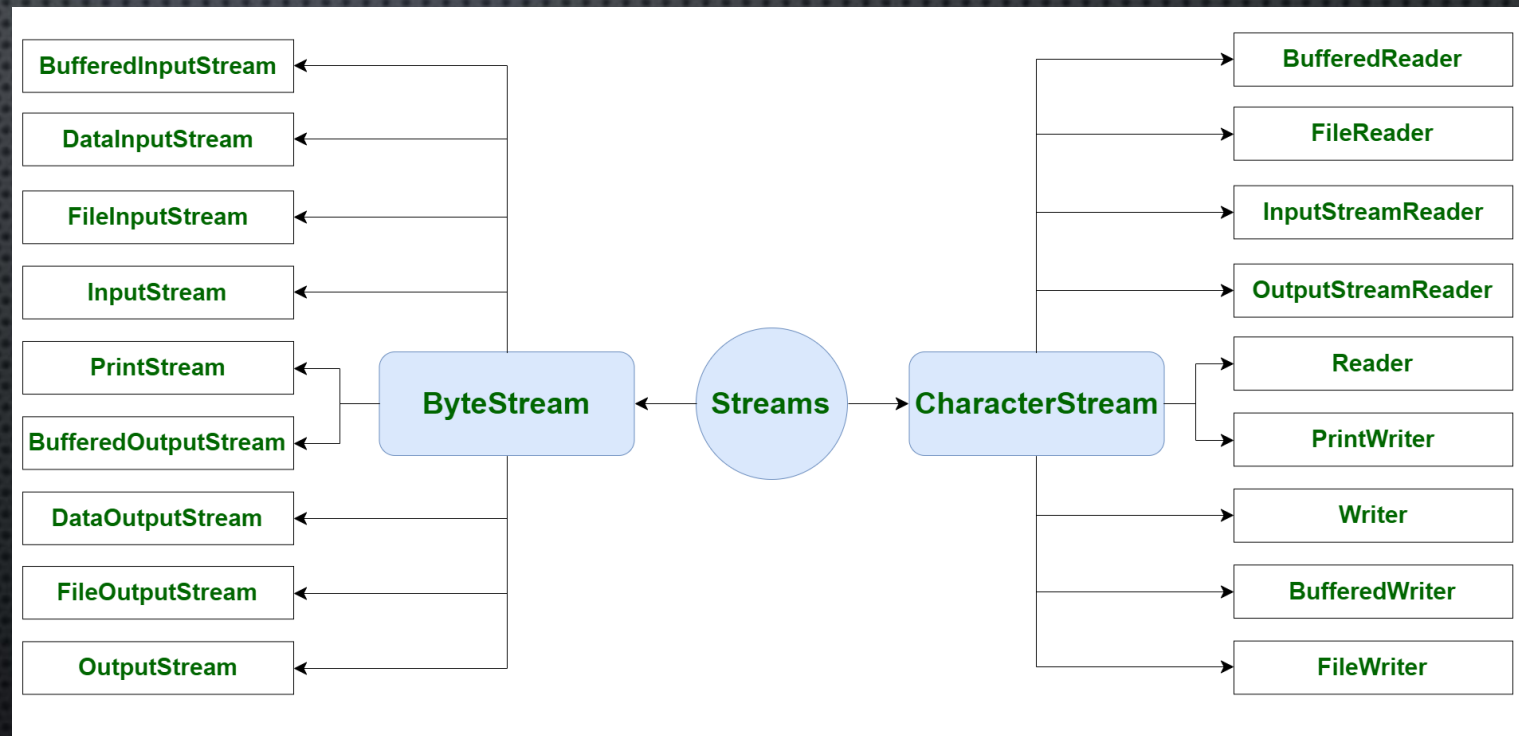
ПОТОЦИ В JAVA

ТИПОВЕ

- ВХОДНО-ИЗХОДНИ (INPUT/OUTPUT (I/O) ПОТОЦИ)
 - ЧЕТЕНЕ ИЛИ ЗАПИС НА ИНФОРМАЦИЯ
 - ФАЙЛОВЕ
 - МРЕЖОВА КОМУНИКАЦИЯ
- JAVA STREAM API

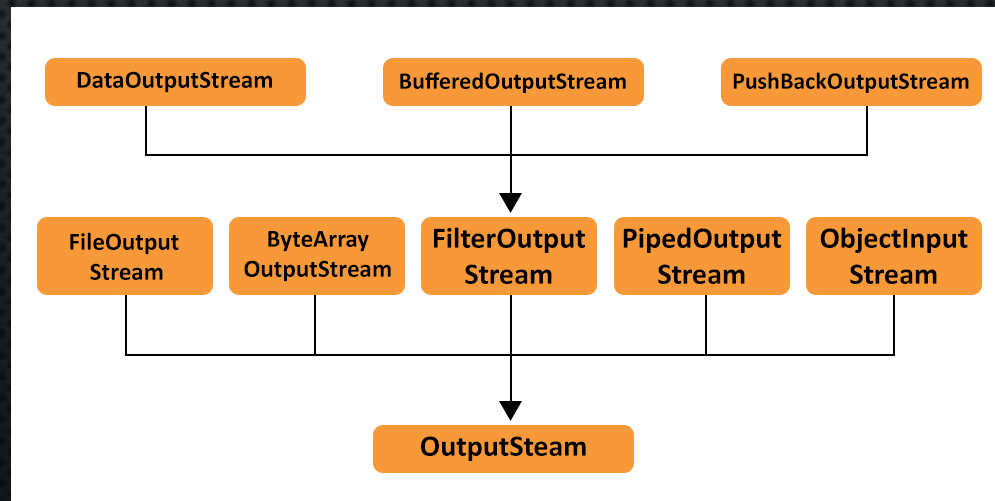
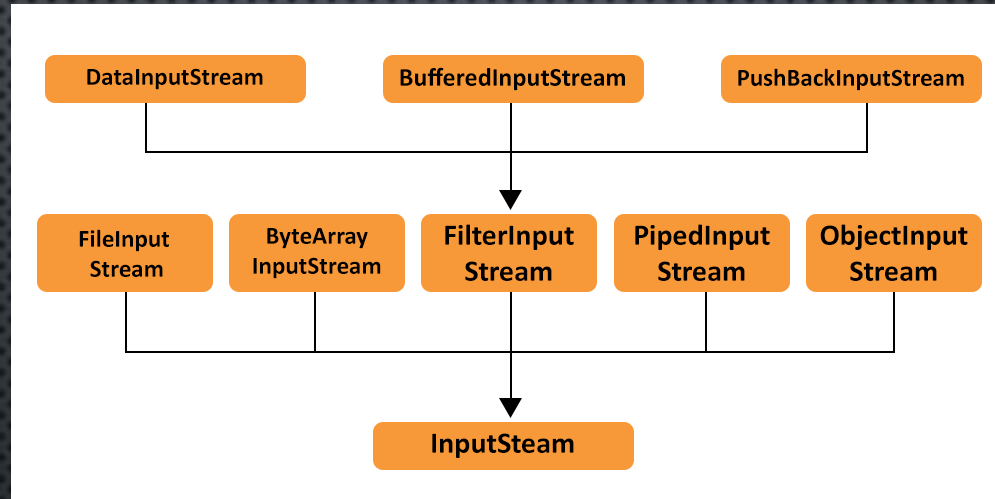
ТИПОВЕ ВХОДНО-ИЗХОДНИ ПОТОЦИ

- БАЙТОВО-ОРИЕНТИРАНИ
- СИМВОЛНО-ОРИЕНТИРАНИ



БАЙТОВО-ОРИЕНТИРАНИ ПОТОЦИ

- ВХОДНИ
- ИЗХОДНИ



БАЙТОВО-ОРИЕНТИРАНИ ПОТОЦИ

- `FileInputStream inputStream = new FileInputStream("file.txt");`
- `FileOutputStream outputStream = new FileOutputStream("output.txt");`

INPUTSTREAMS

- ФУНКЦИОНАЛЬНОСТИ
 - `READ()`, `MARK(INT ARG)`, `RESET()`, `CLOSE()`, `READ(BYTE [] ARG)`, `SKIP(LONG ARG)`, `MARKSUPPORTED()`

INPUTSTREAMS

```
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;

public class FileCopyExample {
    public static void main(String[] args) {
        String inputFile = "SOURCE.TXT";
        String outputFile = "DESTINATION.TXT";

        try (
            FileInputStream inputStream = new FileInputStream(inputFile);
            FileOutputStream outputStream = new FileOutputStream(outputFile)
        ) {
            byte[] buffer = new byte[1024];
            int bytesRead;

            while ((bytesRead = inputStream.read(buffer)) != -1) {
                outputStream.write(buffer, 0, bytesRead);
            }

            System.out.println("Копирането на файла завърши успешно.");
        } catch (IOException e) {
            System.err.println("Възникна грешка: " + e.getMessage());
        }
    }
}
```

OUTPUTSTREAMS

- ФУНКЦИОНАЛЬНОСТИ
 - `FLUSH()`, `CLOSE()`, `WRITE(INT B)`, `WRITE(BYTE [] B)`

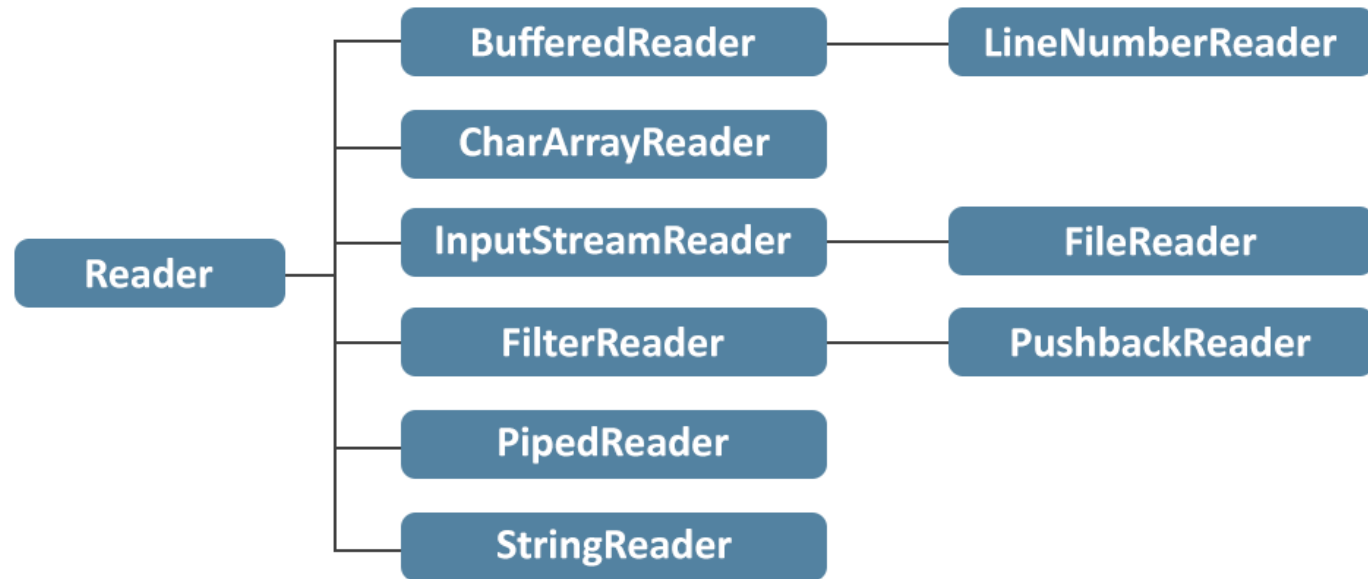
OUTPUTSTREAMS

```
OutputStream output = new FileOutputStream("FILE.TXT");  
    byte[] B = { 79,79,80,49 };  
    output.write(B);  
    output.close();
```

СИМВОЛНО ОРИЕНТИРАНИ ПОТОЦИ READER, WRITER

- РАБОТА С ПОТОЦИ ИНТЕРПРЕТИРАНИ КАТО СИМВОЛИ

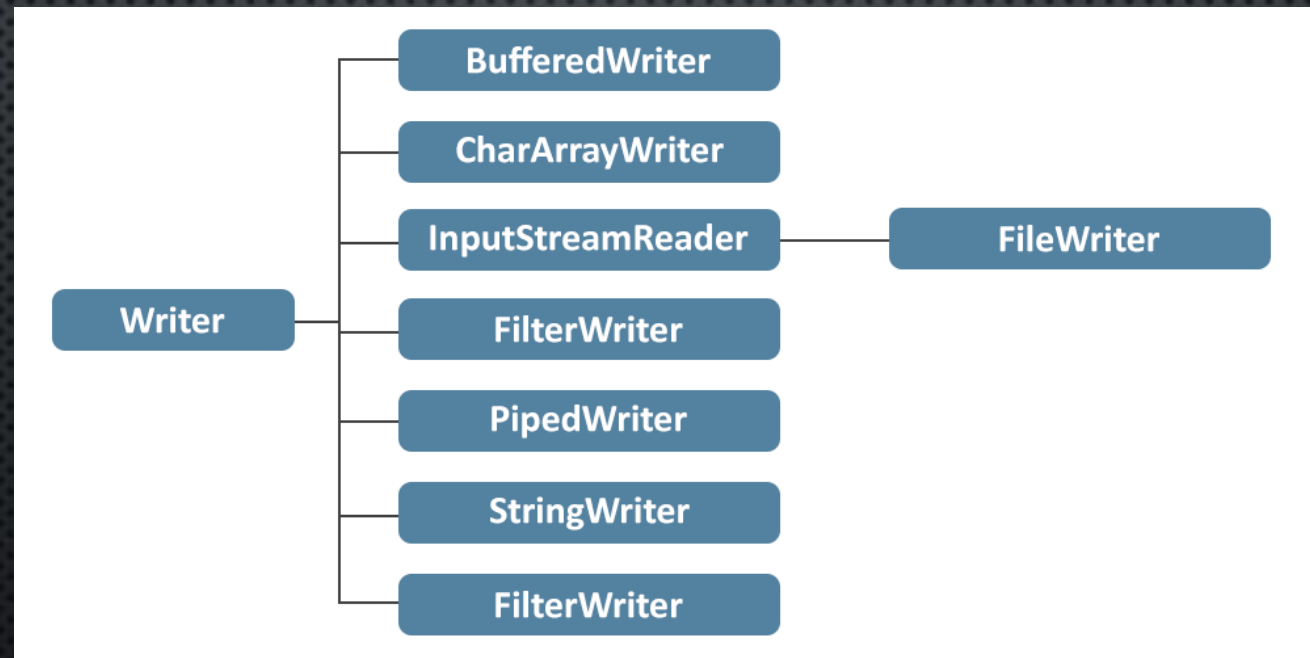
READER



READER

```
CHAR[] BUFFER = NEW CHAR[100];  
READER INPUT = NEW FILEREADER("FILE.TXT");  
SYSTEM.OUT.PRINTLN("DATA AVAILABLE FOR READING? " + INPUT.READY());  
INPUT.READ(BUFFER);  
SYSTEM.OUT.PRINTLN("DATA IN THE STREAM -");  
SYSTEM.OUT.PRINTLN(BUFFER);  
INPUT.CLOSE();
```

WRITER



WRITER

```
STRING DATA = "HELLO OOP1.";
WRITER OUTPUT = NEW FILEWRITER("FILE.TXT");
OUTPUT.WRITE(DATA);
OUTPUT.CLOSE();
```

СТАНДАРТНИ ИЗХОДИ И ВХОД

- SYSTEM.IN
- SYSTEM.OUT
- SYSTEM.ERR

I/O НА ВИСОКО НИВО

- ИЗПОЛЗВАНЕ НА LAYERING ЗА ПОЛУЧАВАНЕ НА ПО ДОБЪР ИНТЕРФЕЙС
- DATAINPUTSTREAM - ИНТЕРПРЕТИРА БАЙТОВЕТЕ В ЧИСЛОВИ ТИПОВЕ
- BUFFERED STREAMS (ЕФИКАСНИ И УДОБНИ)

LAYERING STREAMS/ ВМЪКВАНЕ НА ПОТОЦИ

- `FileInputStream` ЧЕТЕ БАЙТОВЕ ОТ ФАЙЛ.
- `DataStream` КОМПОЗИРА (ДВОИЧНИ ФОРМАТИ) БАЙТОВЕТЕ В ДРУГА ПО ПОЛЕЗНА ФОРМА.
- КОМБИНИРАНЕ НА ФУНКЦИОНАЛНОСТИ
 - `FileInputStream fin = new FileInputStream("EMPLOYEE.DAT");`
`DataStream din = new DataStream(fin);`
`double x = din.readDouble();`

ЗАПИС НА ТЕКСТ

- ЗА ПИСАНЕ НА `STRING` (ЗА ПРЕДПОЧИТАНЕ ПРЕД `CHARS`) ДОБАВЯМЕ ДРУГ LAYER:
 - `PRINTWRITER OUT = NEW PRINTWRITER(NEW FILEWRITER("EMPLOYEE.TXT"));`
`OUT.PRINTLN("ALI BABA");`
`DOUBLE SALARY = 34000;`
`OUT.PRINTLN(SALARY);`
- ИЗПОЛЗВА СЕ КАКТО ПРИ `SYSTEM.OUT` - (`SYSTEM.OUT` Е `PRINTWRITER CLASS`)

СИМВОЛ ЗА КРАЙ НА РЕД

- РАЗЛИЧНИ В РАЗЛИЧНИТЕ ОС
 - WINDOWS “\R\N”
 - UNIX “\N”
 - MAC “\R”
- PRINTLN() АВТОМАТИЧНО ИЗБИРА ПРАВИЛНИЯ
- ВЗЕМАНЕ НА СИСТЕМНИЯ СИМВОЛ
 - SYSTEM.GETPROPERTY(“LINE.SEPARATOR”);

ПРОБЛЕМ

- НЕОБХОДИМОСТ ОТ ПИСАНЕ НА ПОВЕЧЕ КОД ЗА ПОЛУЧАВАНЕ НА УДОБНИ ИНСТРУМЕНТИ ЗА ВХОД (ЧЕТЕНЕ)
- `JAVA.UTIL.SCANNER`

SCANNER

- НАДСТРОЙКА НА ПОТОЦИТЕ
- МНОЖЕСТВО ФУНКЦИОНАЛНОСТИ

SCANNER

```
SCANNER IN = NEW SCANNER(SYSTEM.IN);  
SYSTEM.OUT.PRINT("ENTER YOUR NAME: ");  
STRING NAME = IN.NEXTLINE();  
SYSTEM.OUT.PRINTLN("NAME IS: " + NAME);  
IN.CLOSE();
```

ОБЕКТНО ОРИЕНТИРАН ПОДХОД

- ИСКАМЕ ДА ЗАПИСВАМЕ ОБЕКТИ
- НЕ ВСИЧКИ СА ОТ ЕДИН ТИП
- МОЖЕМ ДА ЗАПИШЕМ:
 - ТИПА НА ОБЕКТА
 - ДАННИТЕ КОИТО ДЕФИНИРАТ ОБЕКТА
- СЕРИАЛИЗАЦИЯ

СЕРИАЛИЗАЦИЯ

- ИНТЕРФЕЙС `SERIALIZABLE`
- `OBJECTOUTPUTSTREAM`, `OBJECTINPUTSTREAM`
 - `WRITEOBJECT()`, `READOBJECT()`

СЕРИАЛИЗАЦИЯ

```
OBJECTOUTPUTSTREAM OUT = NEW OBJECTOUTPUTSTREAM (NEW FILEOUTPUTSTREAM("EMPLOYEE.DAT");  
EMPLOYEE HARRY = NEW EMPLOYEE("HARRY HACKER",50000,1989,10,1);  
MANAGER BOSS = NEW MANAGER("CARL CRACKER",75000,1987,1,13);  
OUT.WRITEOBJECT(HARRY);  
OUT.WRITEOBJECT(BOSS);
```

СЕРИАЛИЗАЦИЯ

- КОГАТО ГИ ЧЕТЕМ ОБРАТНО Е ВАЖНО ДА НАПРАВИМ CAST КЪМ КОРЕКТЕН ДАННОВ ТИП
 - `SERIALVERSIONUID`
- „TRANSIENT“
 - `STATIC`

SERIALVERSIONUID

- СТАТИЧНА КОНСТАНТА ОТ ТИП LONG
 - ИДЕНТИФИЦИРА ВЕРСИЯТА НА КЛАСА ПРИ ДЕСЕРИАЛИЗАЦИЯ

SERIALVERSIONUID

1. СЪВМЕСТИМОСТ МЕЖДУ ВЕРСИИ НА КЛАСА:

1. КОГАТО ОБЕКТ Е СЕРИАЛИЗИРАН, ИНФОРМАЦИЯТА ЗА НЕГОВИЯ **SERIALVERSIONUID** СЕ ЗАПИСВА В БАЙТОВИЯ ПОТОК.
2. ПРИ ДЕСЕРИАЛИЗАЦИЯ JAVA ПРОВЕРЯВА ДАЛИ **SERIALVERSIONUID** НА КЛАСА СЪВПАДА С ТОЗИ НА СЕРИАЛИЗИРАНИЯ ОБЕКТ.
3. АКО НЯМА СЪВПАДЕНИЕ, ВЪЗНИКВА **INVALIDCLASSEXCEPTION**, ЗАЩОТО СИСТЕМАТА ПРЕДПОЛАГА, ЧЕ КЛАСЪТ Е ПРОМЕНЕН И НЕ Е СЪВМЕСТИМ С ДАННИТЕ.

2. КОНТРОЛ ВЪРХУ СЕРИАЛИЗАЦИЯТА:

1. РЪЧНОТО ЗАДАВАНЕ НА **SERIALVERSIONUID** ПОЗВОЛЯВА ДА СЕ ЗАПАЗИ СЪВМЕСТИМОСТ МЕЖДУ РАЗЛИЧНИ ВЕРСИИ НА КЛАСА, ДОРИ АКО СТРУКТУРАТА МУ Е ЛЕКО ПРОМЕНЕНА (НАПРИМЕР ДОБАВЯНЕ НА НОВО ПОЛЕ).

SERIALVERSIONUID

- АКО **SERIALVERSIONUID** НЕ Е ЗАДАДЕН РЪЧНО, JAVA ГО ГЕНЕРИРА АВТОМАТИЧНО, БАЗИРАЙКИ СЕ НА СТРУКТУРАТА НА КЛАСА (ИМЕНА НА ПОЛЕТА, МЕТОДИ, МОДИФИКАТОРИ И Т.Н.).
- ПРИ ВСЯКА ПРОМЯНА В КЛАСА (НАПРИМЕР ДОБАВЯНЕ НА НОВО ПОЛЕ), ГЕНЕРИРАНИЯТ **SERIALVERSIONUID** ЩЕ БЪДЕ РАЗЛИЧЕН, КОЕТО МОЖЕ ДА ДОВЕДЕ ДО ПРОБЛЕМИ С НЕСЪВМЕСТИМОСТ ПРИ ДЕСЕРИАЛИЗАЦИЯ.

ИНТЕРФЕЙС AUTOCLOSEABLE

- TRY С РЕСУРС
- `TRY(FILEOUTPUTSTREAM FILEOUTPUTSTREAM =NEWFILEOUTPUTSTREAM(„OUT.TXT")){`
- `TRY( FILEOUTPUTSTREAM FILEOUTPUTSTREAM =NEW FILEOUTPUTSTREAM(“OUT.TXT”);`
`INPUTSTREAM INPUT = NEW FILEINPUTSTREAM(“IN.TXT”)){`

STREAM API

- STREAM API - JAVA 8
 - ФУНКЦИОНАЛЕН ПОДХОД ЗА ОБРАБОТКА НА КОЛЕКЦИИ ОТ ДАННИ.
- ХАРАКТЕРИСТИКИ НА STREAM API:
 - РАБОТИ С ДАННИ КАТО ПОТОК (STREAM), ВМЕСТО ДИРЕКТНО С КОЛЕКЦИИ.
 - ПОЗВОЛЯВА **ЛЕНИВИ ОПЕРАЦИИ**, КОИТО СЕ ИЗПЪЛНЯВАТ САМО КОГАТО СА НЕОБХОДИМИ.
 - ОПЕРАЦИИТЕ СА РАЗДЕЛЕНИ НА: **ПРОМЕЖДУТЪЧНИ ОПЕРАЦИИ (INTERMEDIATE OPERATIONS)**:
 - ПРИМЕР: FILTER, MAP, SORTED.
 - **КРАЙНИ ОПЕРАЦИИ (TERMINAL OPERATIONS)**:
 - ПРИМЕР: COLLECT, FOREACH, REDUCE.

СЪЗДАВАНЕ НА ПОТОК

```
LIST<INTEGER> NUMBERS = ARRAYS.ASLIST(1, 2, 3, 4, 5);  
STREAM<INTEGER> NUMBERSTREAM = NUMBERS.STREAM();
```

ФИЛТРИРАНЕ

```
LIST<INTEGER> EVENNUMBERS = NUMBERS.STREAM()  
    .FILTER(NUM -> NUM % 2 == 0)  
    .COLLECT(COLLECTORS.TO_LIST());
```

ТРАНСФОРМИРАНЕ

```
LIST<STRING> NAMES = ARRAYS.ASLIST("JOHN", "ALICE", "BOB");  
LIST<STRING> UPPERCASENAMES = NAMES.STREAM()  
    .MAP(String::TOUPPERCASE)  
    .COLLECT(COLLECTORS.TOLIST());
```

СОТИРАНЕ

```
LIST<INTEGER> SORTEDNUMBERS = NUMBERS.STREAM()  
    .SORTED()  
    .COLLECT(COLLECTORS.TO_LIST());
```

ОБОБЩЕНИЕ (REDUCE)

```
INT SUM = NUMBERS.STREAM()  
    .REDUCE(0, INTEGER::SUM);
```

ИТЕРАЦИЯ

```
NUMBERS.STREAM()  
    .FOREACH(SYSTEM.OUT::PRINTLN);
```

РЕФЕРЕНЦИЯ КЪМ МЕТОД - ::

```
LIST<INTEGER> NUMBERS = ARRAYS.ASLIST(1, 2, 3, 4, 5);  
INTEGER.SUM INT SUM = NUMBERS.STREAM()  
    .REDUCE(0, INTEGER::SUM);  
SYSTEM.OUT.PRINTLN("SUM: " + SUM);
```

ЛАМБДА АЛТЕРНАТИВА

```
LIST<INTEGER> NUMBERS = ARRAYS.ASLIST(1, 2, 3, 4, 5);
```

```
    INT SUM = NUMBERS.STREAM()
```

```
        .REDUCE(0, (A, B) -> A + B);
```

```
SYSTEM.OUT.PRINTLN("SUM: " + SUM);
```

ИЗПОЛЗВАНЕ

- **РЕФЕРЕНЦИЯ КЪМ МЕТОД:** ПРОСТО ИЗВИКВАНЕ НА МЕТОД, БЕЗ ДОПЪЛНИТЕЛНА ЛОГИКА.
- **ЛАМБДА ИЗРАЗ:** ДОПЪЛНИТЕЛНА ОБРАБОТКА ИЛИ СЛОЖНОСТ В ЛОГИКАТА.